

Assignment 2: Constraint Satisfaction

CSE 352 Fall 2025

1 Due Date and Collaboration

- The assignment is due on **Oct 18th 23:59 Eastern Time**. You have a total of six extra days for late submission across all the assignments. You can use all six days for a single assignment, two days for each assignment – whatever works best for you. Submissions will be docked 20% for every late day beyond the six-day limit.
- You can collaborate to discuss ideas to understand the concepts and math.
- You should NOT collaborate at the code or pseudo-code level. This includes all implementation activities: design, coding, and debugging.
- You should NOT use any code that you did not write to complete the assignment.
- The homework will be cross-checked. **Do not cheat at all! It's worth doing the homework partially instead of cheating and copying your code and get 0 for the whole homework.**

2 Goals

The main goal of this assignment is to 1) understand the impact of the enhancements to depth-first search with backtracking 2) learn the details of min-conflicts algorithm, which we did not see in class.

In this programming assignment you will build a program that can solve graph coloring problems. Just as a reminder, the graph coloring problem is one where you are given a set of regions (variables), a set of colors (domains of these variables), and the adjacency information for these regions (constraints that connect the variables as a constraint graph).

3 Implementation

Your program will solve the graph coloring problem using (i) Depth-first search with backtracking (DFS-B), (ii) Depth-first search with backtracking and addi-



tional improvements we discussed in class (DFS-B++), (iii) Min-Conflicts Local Search (MCLS), (iv) Min-Conflicts Local Search with Restarts (MCLS-R)

3.1 DFS-B and DFS-B++

DFS-B is the basic version of depth-first search with backtracking.

DFS-B++ will include variable ordering, value order, and forward-checking enhancements.

NOTE: You can also optionally implement the AC3 algorithm instead of forward checking if you want to learn. But forward checking is enough for the assignment.

We discussed all of these in class. If you consult any web resources for the algorithms, please include the URLs to these resources.

3.2 Min-Conflicts Local Search (MCLS)

Min-conflicts Local Search is an algorithm that we did not see in class. It is a simple and fast heuristic method to solve constraint satisfaction problems (CSP). You can find many online resources:

1. https://en.wikipedia.org/wiki/Min-conflicts_algorithm
2. <http://pages.cs.wisc.edu/~bgibson/cs540/handouts/csp.pdf>

One drawback of the Min-Conflicts algorithm is that it might get stuck in a local optimum or run forever without finding a solution, even if one exists.

Termination Criteria In this basic version, ensure that your implementation includes a termination criterion to prevent it from running indefinitely. For example, you can set a maximum limit of 500,000 iterations. This will help avoid cases where the algorithm gets stuck in a local optimum or fails to find a solution within a reasonable time.

If your implementation doesn't find a solution, then it should report a failure message. Your report should document the number of such failures given your termination criteria.

3.3 Min-Conflicts Local Search with Restarts (MCLS-R)

Your solution should handle these cases by using random restarts. In this case, you should log and count the steps taken to solve from the current start state but also the total steps from the beginning including all the abandoned search steps.

Termination Criteria Unless you run restarts forever, Min-Conflicts with restart are not guaranteed to find solutions even when one exists.

As before, make sure you implement some termination criterion, so that your implementation doesn't run forever on a given problem.

If your implementation doesn't find a solution, then it should report a failure message. Your report should document the number of such failures given your termination criteria.

3.4 Sample Problems

Each problem consists of N variables, numbered from 0 to $N - 1$, and M binary constraints connecting them. Each variable can be assigned one of K possible colors, where the domain for all variables is the same.

Input Format: The first line of the input file specifies the values for N , M , and K , separated by a **single** whitespace. Each of the following M lines contains a pair of variables X_i and X_j , also separated by a **single** whitespace. This indicates that X_i and X_j must be assigned different colors. These constraints are unique and do not include self-loops.

Output Format: A valid solution consists of one complete assignment of colors to variables that satisfies all constraints. The output file should contain the assigned color for each variable, one per line.

- The color for variable 0 should appear on line 1, variable 1 on line 2, and so on.
- Colors must be in the range 0 to $K - 1$ as defined in the input file.
- If no valid assignment exists, the output should contain: **No answer.**
- Since multiple valid solutions may exist, any solution that satisfies the constraints is considered correct.

The sample input and output are shown in Table 1:

Input:

- $N = 3$ (3 variables $\rightarrow 0, 1, 2$)
- $M = 3$ (3 constraints $\rightarrow 0-2, 1-0, 0-2$)
- $K = 2$ (2 available colors $\rightarrow \{0, 1\}$)

Input	Output
3 3 2	1
0 2	0
1 0	0
0 2	

Table 1: Example of Input and Corresponding Output

Output:

- 1
- 0
- 0
- Since $N = 3$, the output must contain exactly 3 lines, one for each variable.

Note: The output must strictly follow the above format. Writing only the color values (e.g., ‘m’ per line) is incorrect.

We will provide you with two sets of sample problems (easy and hard).

Below you can find if each sample is solvable or not. **The hard tests are given to optimize your implementation and polish your code, the final tests used for evaluating your codes are going to be easier than the hard ones here.**

backtrack_easy	backtrack_hard	minconflict_easy	minconflict_hard
solvable	solvable	solvable	solvable*

For this assignment, the following expectations apply:

1. The implemented algorithm should complete within **30 seconds** on the provided test cases (**backtrack_easy**, **minconflicts_easy**, and **minconflicts_hard**).
2. The **backtrack_hard** test case is intended to be challenging and may exceed 30 seconds or run indefinitely. If it is not solved within the time limit, that is acceptable.

*: These are solvable in general but **minconflict_hard** samples may not be solvable within the given time, so it is fine if your implementation is not able to handle these problems. “No answer” is also acceptable for these problems.

3.5 Problem Generator Script

You are provided a script to generate more CSP problems. To generate solvable problems use the following command:

```
python CSPGenerator.py <N> <M> <K> <output_file_path>
```

where <N>, <M>, <K> are the same as above. `output_file_path` is output file path for test problem.

Important:

- This script does **not** guarantee that every generated problem will be solvable for the given parameters.
- The generated CSPs are exclusive, meaning the script may fail if M is too large.
- **For DFS-based algorithms (DFSB, DFSB++)**, problems with $M < 200$ generally complete within a few minutes, but as M increases beyond this, the solving time grows exponentially.
- **For Min-Conflicts (with or without restarts)**, problems with $M < 500$ typically solve within a few minutes, but larger values can cause the algorithm to either take significantly longer, get stuck in a local optimum, or run indefinitely without finding a solution.

To ensure your algorithm does not run indefinitely, implement a **termination criterion**. For example, setting a **maximum iteration limit** (e.g., 500,000 iterations) prevents Min-Conflicts from getting stuck indefinitely.

To generate a random problem, which may or may not be solvable, use the following command:

```
python CSPGenerator.py <N> <M> <K> <output_file_path> 0
```

You should only use this command to generate **hard problems** for analysis. These problems are useful for testing algorithm efficiency and handling edge cases. However, keep in mind:

- **DFS-based algorithms** may take significantly longer as M increases, and when M is large, the solving time can become impractical.
- **Min-Conflicts** may either take a long time to solve or run indefinitely without a solution when M is large. Always ensure your implementation includes a termination condition.

You will only use this command to generate hard problems to use for your analyses below.

3.6 Experiments and Analysis

Outputs on Sample Problems Your first task is to run your algorithms on the sample problems given to you. Run all four algorithms on the sample problems and submit the output files.

Suppose your input file is named `filename.txt` and you run the DFS-B algorithm on it. Then your output file should be named `DFS-B-output-filename.txt`.

Problem Sets for Analyses The second task is to conduct an analysis on the different algorithms using problems that you generate.

The goal here is to understand how the different algorithms work on problems of varying levels of difficulty and search space size. Think about how N , M , and K potentially impact the difficulty of the problem and the search space. We want to understand the trends in performance if we varied one of these problem dimensions while keeping the other two fixed.

Conduct the experiments and fill out the table 2. For each such configuration generate 20 problems. Compare the performance of the different algorithms on these problem sets. For each problem set of 20 problems compute the mean and standard deviation of the performance metrics.

Analysis Metrics: Compare the algorithms on the following metrics:

1. **Number of States Explored / Steps Taken:**

- For **DFS-B++**, record the total number of states explored during the search process.
- For **Min-Conflicts Local Search with Restarts (MCLS-R)**, track the number of steps taken until the algorithm either finds a solution or reaches the termination condition.

2. **Wall-Clock Execution Time:**

- Measure the total execution time of each algorithm using Python's built-in `time` module.
- Use `time.perf_counter()` for high-precision time measurement in milliseconds.

3. **Peak Memory Usage:**

- Track the peak memory usage for **DFS-B++** and **Min-Conflicts with Restarts**.
- Use Python's `sys.getsizeof()` function to measure the memory consumption of key data structures, such as the open and closed lists in DFS and the state representation in Min-Conflicts.

4. **Success Percentage:** The percentage of problems for which a solution was found.

For each algorithm, conduct experiments on multiple problem instances with varying sizes (N , M , K). Report the results as the **mean and standard deviation** over multiple runs. Clearly present your findings in a table and analyze the observed performance differences.

Please report your results formatted as a table with the structure shown below.

N	M	K	DFS-B++		Min-Conflicts with Restart	
			States Explored	Time (ms)	Steps Taken	Time (ms)
20	100	4	mean±std	mean±std	mean±std	mean±std
50	625	4				
100	2500	4				
200	10000	4				
400	40000	4				

Table 2: Comparison of DFS-B++, and Min-Conflicts with Restart across different problem sizes.

3.7 Requirements

1. You must implement the algorithms from scratch in **Python 3.7+**.
2. You CANNOT use external libraries that provide search related capabilities directly.
3. You CAN use native python libraries that provide support for data structures (e.g., priority [queues](#)) but your program should run on its own.
4. You CANNOT use existing implementations of any kind for development or reference.
5. You CAN use pseudo-code or descriptions of the algorithms for reference but not actual code.
6. You CAN discuss with your friends about which algorithms to use and to understand the algorithms but not share or discuss code.
7. All code will be tested by running through plagiarism detection software.
8. You should include cite any web resources or friends you used/discussed with for pseudo-code or the algorithm itself. Failure to do so will result in an F.
9. Your algorithm should be documented at a method level briefly so they can be read and understood by the TAs.

4 What should you turn in?

A single ZIP file containing the following files:

- README - Code Implementation details. Please include your Python version or any other information that is necessary for TAs to run your code.
- Code - You should turn in your source code. Your source code can be structured however you like but it needs to have two application files: **dfsb.py** and **minconflicts.py**. Your solvers should be able to take the input file specified in the format above and output a coloring consistent with the constraints.
 1. **dfsb.py** – This should run in two modes. a) Plain DFS-B and b) DFS-B with *variable ordering*, *value ordering* and *forward checking*. A sample command for executing dfsb.py should be as below:

```
python dfsb.py <INPUT_FILE> <OUTPUT_FILE>
               <MODE_FLAG>.
```

<MODE_FLAG> can be either 0 (plain DFS-B) or 1 (improved DFS-B++).

2. **minconflicts.py** – This should run in two modes. a) Plain MinConflicts (MCLS) and b) MinConflicts with *Restarts* (MCLS-R).

You may have to adjust the MinConflicts algorithm to avoid plateaus or getting stuck in local depressions.

A sample execution of minconflicts.py should be as below:

```
python minconflicts.py <INPUT_FILE> <OUTPUT_FILE>
                      <MODE_FLAG>.
```

<MODE_FLAG> can be either 0 (plain MCLS) or 1 (improved MCLS-R).

- Four output files - You should turn in the output, named *input_file-output.txt*, of your program on the four input files shipped as part of the assignment.

The backtrack_* files are to be used as inputs to dfsb.py in the improved mode.

The minconflict_* files are to be used as inputs to minconflicts.py.

e.g., for *backtrack_easy* as input you should output in a *backtrack_easy-output.txt* file.

- Report (PDF) – This should be a 3-4 page report, named SBUID-LastName-FirstName-A2.pdf, which includes three sections:

1. Briefly explain how each method works including pseudo code for DFS-B, DFS-B++, MCLS, and MCLS-R.
2. Tables describing the performance of the algorithms (DFSB, DFSB++, MCLS, and MCLS-R) on your generated problems.
3. Explain the observed performance differences in terms of what you know about the different algorithms and the types of problems you have generated.

Please put all these files in the same folder. Your submission should be one zip file named **SBUID-LastName-FirstName-A2.zip**, containing all source code and report under a single folder named **SBUID-LastName-FirstName-A2**, with no other folders in the zip file. For example, if you are Jane Doe and your SBU ID is 116612345, then you **MUST** name the folder **116612345-Doe-Jane-A2** and zipped folder should be **116612345-Doe-Jane-A2.zip** Please note that the file name for the source codes **MUST** be exactly the same as instructed.

5 Grading

Here is a coarse-grained grading rubric.

- DFSB implementation (40 points)
 1. As part of the evaluation of your implementation, your code will be run against several test cases. The time limit for solving these tests is 30 seconds. The limit is not strict, which means that we will select test cases our own code can solve in no more than 30 seconds.
 2. Detail comments in source codes (10 points)
 - Comments that serve as code documentation for readers.
 - Explain the purpose of lines/functions in the code.
 - No need to have comments on every single line but main steps that explain what you are doing.
- MinConflicts implementation (40 points) – Same as above.
- Report (20 points).

-5 points if the submission is not in the specified format.